Comunicación entre componentes

# Tema 4. Unidad 2

## Ejercicios

Universitat Oberta de Catalunya

## Index

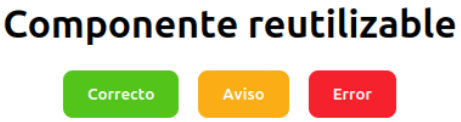

Ejercicio 1: Componente mensajes reutilizable 2
Ejercicio 2: Registro usuarios 3
Ejercicio 3: Música 7

# Ejercicio 1: Componente mensajes reutilizable

El objetivo de este ejercicio es crear un componente para mostrar mensajes que sea reutilizable. Para ello definiremos una interfaz para los datos. Como hemos visto crearemos un carpeta models dentro de src. El nombre del fichero serà alert-props.interface.ts con las siguientes propiedades:

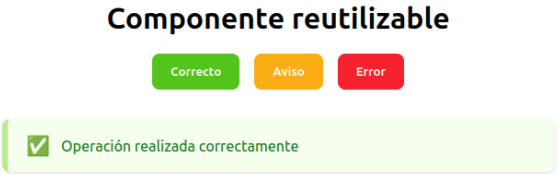
- message: string (obligatorio)
- type: ‘success’ | ‘warning’ | ‘error’ | ‘info’ (opcional)
- duration: number (opcional, en milisegundos)

También debemos crear el componente reutilizable *AlertComponent.vue* dentro de la carpeta *components*. A continuación, crearemos tres botones, como los que se muestran en la imagen T4U2EJ1_IMG1, de manera que al hacer clic en cada botón se muestre el componente *AlertComponent*. En cada caso, se deberá pasar un tipo (type) y un mensaje (message) diferente, según el botón que se haya pulsado.




T4U2EJ1_IMG1

En la siguiente imagen(T4U2EJ1_IMG2) podemos ver el comportamiento al hacer clic en el botón “Correcto”.




T4U2EJ1_IMG2

En la imagen(T4U2EJ1_IMG3) podemos ver el comportamiento al hacer clic en el botón “Aviso”.

T4U2EJ1_IMG3

En la siguiente imagen(T4U2EJ1_IMG4) podemos ver el comportamiento al hacer clic en el botón “Error”.

T4U2EJ1_IMG4

AlertComponent gestiona su propia visibilidad interna (por ejemplo, una variable visible interna) y, cuando se hace invisible, el componente sigue montado dentro del padre (App.vue).
Para poder gestionar la visibilidad podemos hacer que el AlertComponent emita un evento cuando ha terminado su tiempo de visibilidad y está listo para ser desmontado (por ejemplo, *emit('close')*). Entonces, el padre escucha @close="alertVisible = false". Esto nos ayudará a centralizar la lógica de cierre en el propio componente hijo.

# Ejercicio 2: Registro usuarios

Tenemos que crear un formulario de registro simple donde aplicaremos los conceptos de comunicación entre componentes, tipado mediante las interfaces y el uso de composables para reutilizar la lógica de validación de nuestro formulario. Para ello tendremos que crear dos componentes App.vue (principal) y el SignUpForm.vue (el hijo). El componente principal se tiene que encargar de gestionar el estado final del registro mostrando un mensaje si el usuario se registra. El componente hijo será donde tendremos los campos del formulario (nombre, correo electrónico y contraseña) junto con un botón para simular el envío de los datos.

T4U2EJ2_IMG1

El estado inicial del formulario como podemos ver en la imagen T4U2EJ2_IMG1, muestra un mensaje de “Pendiente de registro” con un estilo, gestionado por el componente padre. También está el botón de "Regístrate" deshabilitado hasta la correcta introducción de los datos, gestionado por el componente hijo.

Tendremos que validar los campos del formulario. Para ello, utilizaremos composables, por lo que será necesario crear un archivo con todas las validaciones.
El campo nombre debe tener al menos 2 caracteres, el correo electrónico debe tener un formato válido, y la contraseña debe tener una longitud mínima de 8 caracteres. Los mensajes de validación se mostrarán debajo de cada campo (ver imagen T4U2EJ2_IMG2 y T4U2EJ2_IMG3) y también en una zona de errores al final del formulario. Estos mensajes se actualizan en el momento que el usuario introduce o modifica algún dato.

T4U2EJ2_IMG2

T4U2EJ2_IMG3

Cuando el usuario haga clic en el botón de registrarse el componente hijo debe enviar los datos recogidos al padre para mostrar un mensaje de registro correcto, dando la bienvenida con el nombre introducido por el usuario (ver imagen T4U2EJ2_IMG4). Junto a este mensaje tiene que aparecer un botón para volver habilitar el formulario de registro (Nuevo registro) y borrar los datos del formulario, de lo contrario el botón tiene que estar deshabilitado como podemos ver en la imagen T4U2EJ2_IMG4.

T4U2EJ2_IMG4

# Ejercicio 3: Música

Vamos a crear una aplicación que muestre un catálogo de música con un filtro de éxitos. El usuario debe poder alternar entre ver todas las canciones o solo las que son éxitos actuales. El resultado final debe ser similar al de la imagen T4U2EJ3_IMG1.

**Requisitos**:

- Mostrar listado completo de canciones.
- Botón para filtrar las canciones que son exitos.
- Contador que muestre cuántas canciones se están visualizando y el total de canciones.
- Componentes reutilizables para canción.
- Interface Songs en carpeta models para definir la estructura de datos.
- Estilos con scoped CSS en cada componente.

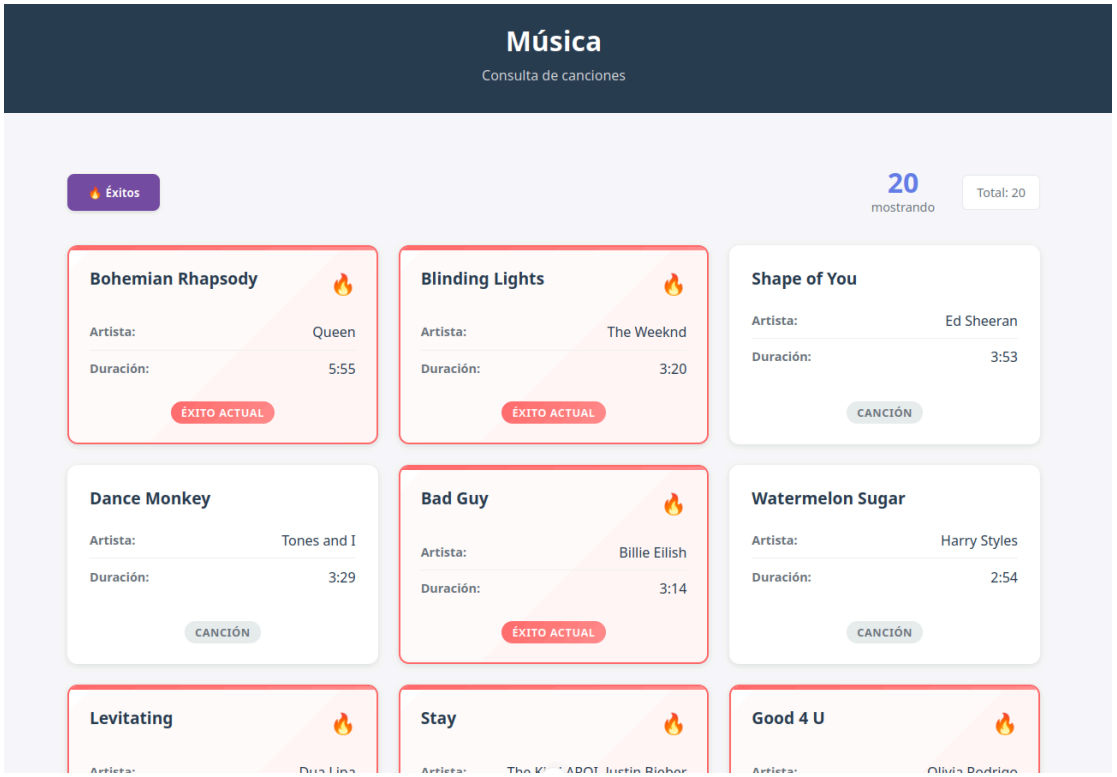



Imagen: T4U1EJ3_IMG1

**Funcionamiento**:

Al hacer clic en el botón *"Éxitos"*, el listado se filtra y únicamente se muestran aquellas que cumplen la condición, como podemos ver en la imagen T4U2EJ3_IMG2. Una vez aplicado el filtro, el texto del botón cambia de *"Éxitos"* a *"Mostrar todas"*.
Si se vuelve a hacer clic, el listado debe restablecerse y mostrar de nuevo todas las canciones.

Además, añadiremos un contador de películas que indique cuántos elementos se están mostrando.

En el estado inicial, el contador debe mostrar el texto:

- *"Mostrando 20"* y al lado *"Total: 20"*, tal como se observa en la imagen T4U2EJ3_IMG1.
- Al aplicar el filtro con el botón *"Éxitos"*, el contador debe actualizarse y mostrar el el total de canciones que se están mostrando y al lado el total ver en la imagen T4U2EJ3_IMG2.

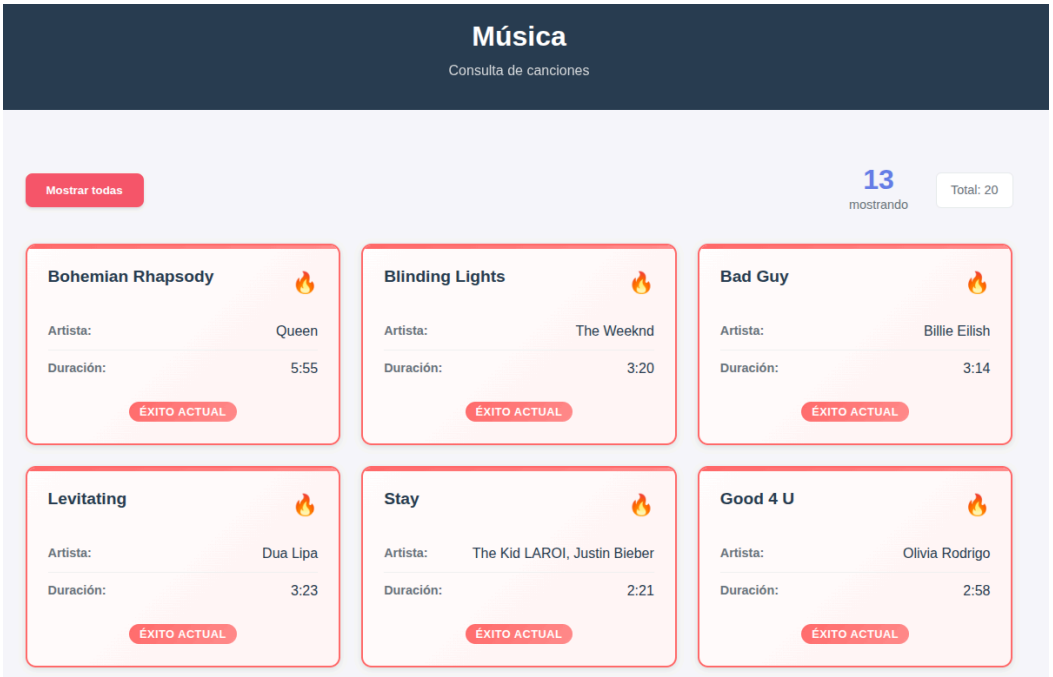



Imagen: T4U1EJ3_IMG2

**Estructura**:

Siguiendo los pasos trabajados en la unidad uno, vamos a crear un proyecto con Vue utilizando el **scaffolder** que nos generará el **boilerplate** del proyecto.
Al crear el proyecto tenemos que seleccionar la característica **Typescript** que nos permitirá trabajar con diferentes tipos de datos.

Una vez creado el proyecto tendremos que crear una estructura como la que podemos ver en la imagen T4U1EJ3_IMG3, donde:

- **components:** definiremos cuatro componentes (Header.vue, SongListContainer.vue,SongItem.vue y Footer.vue)
- **data:** estará el fichero songs.json con los datos de nuestra aplicación (podéis encontrar al final del ejercicio)
- **models:** estará nuestro modelo de datos con los tipos definidos (song.interface.ts)

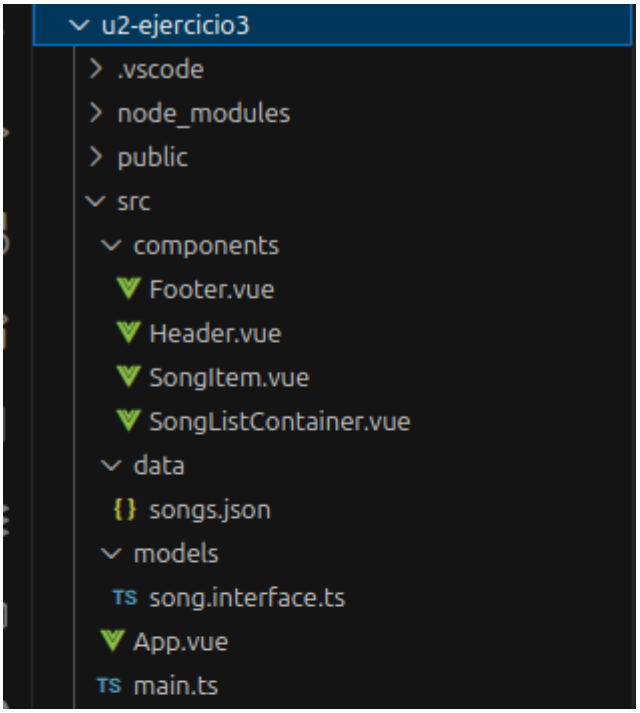



Imagen: T4U2EJ3_IMG3

Los distintos componentes creados (Header, SongListContainer, SongItem y Footer) tienen que contener la información como vemos en la imagen T4U2EJ3_IMG4.

Imagen: T4U1EJ3_IMG4

Del fichero de datos que tenéis que crear (songs.json) con los datos que podéis encontrar al final del ejercicio tenéis que mapear los datos según veis en la imagen T4U1EJ3_IMG5. Podéis ver los campos que deben mostrarse, correspondientes a las distintas propiedades, junto con el botón para filtrar información y el contador de elementos.

Imagen: T4U1EJ3_IMG5

**Datos de ejemplo:**


```
[
  {
    "id": 1,
    "title": "Bohemian Rhapsody",
    "artist": "Queen",
    "duration": "5:55",
    "isTopHit": true
  },
  {
    "id": 2,
    "title": "Blinding Lights",
    "artist": "The Weeknd",
    "duration": "3:20",
    "isTopHit": true
  },
  {
    "id": 3,
    "title": "Shape of You",
    "artist": "Ed Sheeran",
    "duration": "3:53",
    "isTopHit": false
  },
  {
    "id": 4,
    "title": "Dance Monkey",
    "artist": "Tones and I",
    "duration": "3:29",
    "isTopHit": false
  },
  {
    "id": 5,
    "title": "Bad Guy",
    "artist": "Billie Eilish",
    "duration": "3:14",
    "isTopHit": true
  },
  {
    "id": 6,
    "title": "Watermelon Sugar",
    "artist": "Harry Styles",
    "duration": "2:54",
    "isTopHit": false
  },
  {
```

```
  "id": 7,
  "title": "Levitating",
  "artist": "Dua Lipa",
  "duration": "3:23",
  "isTopHit": true
},
{
  "id": 8,
  "title": "Stay",
  "artist": "The Kid LAROI, Justin Bieber",
  "duration": "2:21",
  "isTopHit": true
},
{
  "id": 9,
  "title": "Good 4 U",
  "artist": "Olivia Rodrigo",
  "duration": "2:58",
  "isTopHit": true
},
{
  "id": 10,
  "title": "Save Your Tears",
  "artist": "The Weeknd",
  "duration": "3:35",
  "isTopHit": false
},
{
  "id": 11,
  "title": "Astronaut In The Ocean",
  "artist": "Masked Wolf",
  "duration": "2:12",
  "isTopHit": false
},
{
  "id": 12,
  "title": "Kiss Me More",
  "artist": "Doja Cat ft. SZA",
  "duration": "3:28",
  "isTopHit": true
},
{
  "id": 13,
  "title": "Montero",
  "artist": "Lil Nas X",
```

```
  "duration": "2:00",
  "isTopHit": true
},
{
  "id": 14,
  "title": "Peaches",
  "artist": "Justin Bieber ft. Daniel Caesar, Giveon",
  "duration": "3:18",
  "isTopHit": true
},
{
  "id": 15,
  "title": "Butter",
  "artist": "BTS",
  "duration": "2:42",
  "isTopHit": true
},
{
  "id": 16,
  "title": "Stay With Me",
  "artist": "Sam Smith",
  "duration": "2:52",
  "isTopHit": false
},
{
  "id": 17,
  "title": "Uptown Funk",
  "artist": "Mark Ronson ft. Bruno Mars",
  "duration": "4:30",
  "isTopHit": true
},
{
  "id": 18,
  "title": "Shallow",
  "artist": "Lady Gaga, Bradley Cooper",
  "duration": "3:37",
  "isTopHit": false
},
{
  "id": 19,
  "title": "Don't Start Now",
  "artist": "Dua Lipa",
  "duration": "3:03",
  "isTopHit": true
},
```

```
  {
    "id": 20,
    "title": "Flowers",
    "artist": "Miley Cyrus",
    "duration": "3:20",
    "isTopHit": true
  }
]
```